

Practical-2

Aim:

To Implementation and Time analysis of linear and binary search algorithm.

Algorithm:

Linear Search:

Start from the first element
Compare each element with the key
If found $\rightarrow$ return position
If not $\rightarrow$ Move to next
end after last element (not found).

Binary Search:

work on a sorted array
Set low and high pointers.
find $mid = (low + high) / 2$
If $key = mid \rightarrow$ return position
If $key < mid \rightarrow$ Search left half
If $key > mid \rightarrow$ Search right half
Repeat until $low > high$ (not found).

Code:

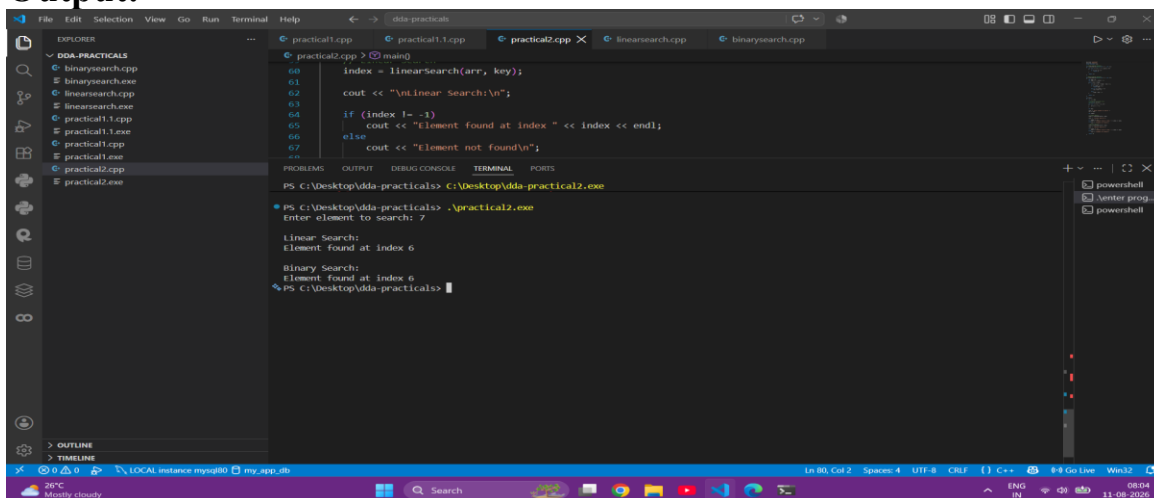
```
#include <iostream>
#include <vector>
using namespace std;
// Linear Search Function
int linearSearch(vector<int> arr, int key) { for
    (int i = 0; i < arr.size(); i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}
// Binary Search Function
int binarySearch(vector<int> arr, int key) {
    int low = 0;
    int high = arr.size() - 1; while
    (low <= high) {
```

```

int mid = low + (high - low) / 2; if
(arr[mid] == key)
    return mid;
else if (arr[mid] < key) low
    = mid + 1;
else
    high = mid - 1;
}
return -1;
}
int main() {
    int n = 100; // smaller size for easy testing
    vector<int> arr(n);
    // Creating a sorted array for
    (int i = 0; i < n; i++)
arr[i] = i + 1;
    int key;
    cout << "Enter element to search: "; cin
>> key;
    int index;
    // Linear Search
    index = linearSearch(arr, key);
    cout << "\nLinear Search:\n"; if
(index != -1)
        cout << "Element found at index " << index << endl; else
        cout << "Element not found\n";
    // Binary Search
    index = binarySearch(arr, key);
    cout << "\nBinary Search:\n"; if
(index != -1)
        cout << "Element found at index " << index << endl; else
        cout << "Element not found\n";
    return 0;
}

```

Output:



```

PS C:\Desktop\dda-practicals> .\practical2.exe
Enter element to search: 7
Linear Search:
Element found at index 6
Binary Search:
Element found at index 6
PS C:\Desktop\dda-practicals>

```

Time complexity:

Linear Search:

Time Complexity $\rightarrow$ time to search in array size n
 $T(n) = T(n-1) + O(1)$

$$T(n) = T(n-1) + 1 = T(n-2) + 1 + 1 = \dots T(1) + (n-1)$$

Best Case : $T(1) = O(1)$

final: $T(n) = O(n)$

Best Case: $O(1)$

Worst Case: $O(n)$

Average Case: $O(n)$ Binary Search:

Time Complexity: Let $T(n)$ = Time to search in array of size n . For checking middle element constant $O(1)$ search half the array $\rightarrow n/2$

$$T(n) = T\left(\frac{n}{2}\right) + 1 = T\left(\frac{n}{4}\right) + 1 + 1 = \dots T(1) + \log_2 n$$

best case: $T(1) = O(1)$

$$T(n) = O(\log n)$$

best case = $O(1)$

Average Case: $O(\log n)$

Worst case: $O(\log n)$

Result:

Thus, Linear Search and Binary Search were successfully implemented and executed. Linear Search checks each element one by one, while Binary Search efficiently searches for a sorted array by repeatedly dividing the search range into two halves. Binary Search is faster than Linear Search for large, sorted arrays.